

CoinData Platform

Upbit · Binance의 실시간 시세와 환율을 수집해 김치 프리미엄, 캔들, 기술 지표를 계산하고 REST·SSE로 제공하는 실시간 시장 데이터 파이프라인.

23

GRADLE MODULES

13

KAFKA TOPICS

5

PIPELINE STAGES

Java 21

SPRING BOOT 3.4

01. 프로젝트 개요

CoinData는 한국 거래소(Upbit)와 해외 거래소(Binance)의 실시간 가격을 비교해 **김치 프리미엄**을 계산하고, 캔들·기술 지표 분석까지 제공하는 백엔드 시스템입니다. 실시간 시세 처리, 시계열 분석, REST/SSE API 제공의 세 가지 기술 영역을 하나의 파이프라인으로 통합했습니다.

왜 만들었는가

국내·해외 거래소 가격 차이를 실시간으로 추적하고, 그 위에 기술적 분석을 얹어 화면 단까지 끊임 없이 흐르는 데이터 파이프라인을 직접 만들어보고 싶었습니다. Kafka·Redis·시계열 DB·SSE 같은 분산 시스템 컴포넌트를 "왜 필요한가"에서 출발해 실제 코드로 결합하는 경험을 목표로 잡았습니다.

핵심 기능

- **실시간 시세 수집** — Upbit · Binance WebSocket을 받아 Kafka로 정규화 발행
- **김치 프리미엄 계산** — 환율 동기화, 비교 거래소 호가, 트레이드오프 명시한 정밀도 처리
- **캔들 · 기술 지표 분석** — 파티션 단위 상태 관리, Redis state 복원, EMA·RSI·STDDEV 계산
- **조회 API + 실시간 SSE** — RFC 7807, cursor/offset 페이지징, Reactor Sinks 기반 fanout
- **경제지표 수집** — FRED API + Investing/Yahoo 크롤링, 동적 스케줄러
- **관측성** — Prometheus · Grafana 스택, 모듈 self-heartbeat Kafka 발행

TABLE OF CONTENTS

01.	프로젝트 개요	P. 2
02.	기술 스택 · 빌드 환경	P. 3
03.	전체 아키텍처 · 모듈 의존성	P. 4
04.	데이터 흐름 · Kafka 토폴로지	P. 5
05.	Hexagonal Architecture 적용	P. 6
06.	핵심 설계 결정 5선 (기술)	P. 7-10
07.	협업 · 범위 관리 결정 (비기술)	P. 11
08.	코딩 컨벤션 · 코드 품질	P. 12
09.	테스트 전략	P. 13
10.	한계 · 학습 · 향후 작업	P. 14
11.	차별점 요약 · 마무리	P. 15

02. 기술 스택 · 빌드 환경

LANGUAGE & FRAMEWORK

- Java 21 (record · sealed · pattern matching 활용)
- Spring Boot 3.4.5
- Spring Cloud 2024.0.1
- Spring Modulith 1.3.5
- Reactor (Sinks.Many, Flux)

BUILD & MODULE

- Gradle 8.4 (멀티모듈, 23개 sub-project)
- Lombok (생성자 주입 + record 보완)
- QueryDSL 5.x (Read-side)
- Jackson + DSL-JSON / Jsoniter (고성능)

MESSAGING & STORAGE

- Apache Kafka (JSON record 직렬화)
- PostgreSQL + TimescaleDB (시계열 hypertable)
- Redis (latest 캐시 + Analytics state 복원)

OBSERVABILITY & TEST

- Micrometer + Prometheus + Grafana
- Spring Actuator (/actuator/prometheus)
- JUnit 5 · BDD Mockito · AssertJ · Awaitility
- Testcontainers · WireMock · @EmbeddedKafka

검증된 빌드 명령

문서 갱신 시점(2026-05-13)에 다음 두 명령이 통과한 상태로 관리합니다. 전체 test 는 Kafka · PostgreSQL · Redis · Testcontainers 의존이 있어 모듈별로 분리해서 실행합니다.

```
# 컴파일 검증
.\gradlew.bat compileJava
.\gradlew.bat compileTestJava

# 모듈별 테스트
.\gradlew.bat :market_data:test
.\gradlew.bat :analytics:test
.\gradlew.bat :api:test
```

런타임 실행 환경

PowerShell 스크립트 start-runtime.ps1 로 Kafka · PostgreSQL · Redis · Prometheus · Grafana를 Docker Compose로 띄우고, API 인스턴스를 다중으로 실행할 수 있도록 구성했습니다.

```
# 단일 인스턴스
.\scripts\run\start-runtime.ps1

# 멀티 인스턴스 (SSE fanout 검증용)
.\scripts\run\start-runtime.ps1 -ApiInstances 2 -ApiBasePort 8080

# 전체 (ingestion + economic 포함)
.\scripts\run\start-runtime.ps1 -All
```

환경 변수 외부화

민감 정보와 외부 의존성 endpoint는 application.yml 에 직접 노출하지 않고 환경 변수로 fallback 처리했습니다.

- KAFKA_BOOTSTRAP_SERVERS , KAFKA_SERVERS
- REDIS_HOST , REDIS_PORT
- JPA_DDL_AUTO (개발/운영 분리)
- 거래소별 *_BASE_URL , *_WS_URL (Upbit · Binance · Naver FX)
- FRED API key는 application-test.yml 에서도 placeholder 처리

03. 전체 아키텍처 · 모듈 의존성

23개 Gradle 모듈을 **write-side**(쓰기/계산), **read-side**(조회), **API**(진입점), **인프라/계약 공통**으로 분리했습니다. 의존 방향은 단방향이며, `query/*`의 write-side 역참조와 `api`의 도메인 모듈 직접 의존을 금지합니다.

모듈 그룹

그룹	모듈	역할
계약 · 공통	<code>contracts</code> , <code>infra_shard</code> , <code>infra_heartbeat</code>	Kafka 메시지 record, 공통 mapper 8종, Redis 키 유틸, 모듈 health 전파
거래소 클라이언트	<code>infra_exchange/upbit</code> , <code>infra_exchange/binance</code>	REST · WebSocket client, DTO, 인증 필터, properties (비즈니스 usecase 금지)
수집 (Ingestion)	<code>ingestion/exchange/*</code> , <code>fx_ingestion</code>	Tick · MarketCode raw publish, Naver FX 1초 스케줄러
경제지표	<code>economic/economic_ind/*</code>	FRED API + Investing/Yahoo 크롤링, 동적 스케줄러, DB + Kafka relay
Write-side	<code>meta_data</code> , <code>market_data</code> , <code>analytics</code>	마스터 데이터, Tick/Premium 계산, Candle/Indicator 시계열 분석
Read-side	<code>query/*</code> (4종)	<code>market_data_query</code> , <code>meta_data_query</code> , <code>analytics_query</code> , <code>economic_query</code>
진입점	<code>api</code>	REST controller, composition, Kafka stream consumer + SSE fanout

모듈 의존 방향

의존성은 단방향으로 흐르며, ArchUnit 가드로 회귀를 막을 수 있도록 룰을 권장 항목으로 정의했습니다.

```
contracts
├─ infra_shard
│   └─ infra_exchange/* (client only)
│       └─ ingestion/* (publish only)
│           └─ meta_data, market_data, analytics (write-side)
│               └─ query/* (read-side, no back-reference)
│                   └─ api (query + stream only)
```

의존성 제약 사항

- **RULE** `query/*`는 `market_data`, `analytics`, `meta_data`, `economic_*`에 역참조 금지
- **RULE** `api`는 `query/*`, `contracts`, `infra_shard`, `infra_heartbeat`만 의존
- **RULE** `infra_exchange/*`는 client · DTO · properties 전용, 비즈니스 usecase 금지
- **PROPOSED** ArchUnit 가드로 위 제약을 컴파일 타임 가까이 검증 (권장 단계)

04. 데이터 흐름 · Kafka 토폴로지

End-to-End 파이프라인

Exchange WebSocket (Upbit, Binance)



ingestion-exchange.tick-raw · ingestion-exchange.market-code-raw · ingestion-fx.fx



meta_data (MarketCode DB · meta-data.* publish)

market_data (Tick · Fx 캐시 · Premium 계산 · DB flush)



market-data.tick · market-data.premium · market-data.premium-detail



analytics (파티션별 Candle/Indicator state · Redis snapshot · DB flush)



analytics.tick-candle · analytics.premium-candle · analytics.*-indicator



api (query REST + Kafka consumer + Sinks.Many + SSE fanout)



coin_front (별도 Next.js 프로젝트, REST + SSE 구독)

Kafka 토픽 카탈로그

Topic	Producer	Consumer
ingestion-exchange.tick-raw	ingestion_exchange_shard	market_data
ingestion-exchange.market-code-raw	ingestion_exchange_shard	meta_data
ingestion-fx.fx	fx_ingestion	market_data
meta-data.exchange	meta_data	market_data
meta-data.market-code	meta_data	market_data
market-data.tick	market_data	analytics , api SSE
market-data.premium	market_data	analytics , api SSE
market-data.premium-detail	market_data	analytics , api SSE
analytics.tick-candle	analytics	api SSE
analytics.premium-candle	analytics	api SSE
analytics.tick-indicator	analytics	api SSE
analytics.premium-indicator	analytics	api SSE
economic-ind.indicator	economic_ind_shard	(MVP downstream 없음)
{module}.heartbeat , {module}.health-change	infra_heartbeat	infra_heartbeat

토픽명 변경 시 producer · consumer · NewTopic config · 테스트 assertion · coin_front SSE 이벤트명까지 함께 추적합니다.

05. Hexagonal Architecture 적용

모든 도메인 서비스 모듈은 **Port & Adapter** 구조를 따릅니다. UseCase 인터페이스(port/in)와 외부 의존성 인터페이스(port/out)를 분리해서 Kafka·Redis·PostgreSQL 같은 인프라가 도메인 코드에 새어들어가지 않게 했습니다.

레이어 구조

```
application/
├── port/
│   ├── in/      ← UseCase 인터페이스 (외부 → 도메인)
│   └── out/     ← Port 인터페이스 (도메인 → 외부)
└── usecase/     ← UseCase 구현체 (@Service)

domain/
├── domain/      ← record · enum · sealed interface · 순수 객체
├── service/     ← 도메인 서비스 (계산 로직)
├── buffer/      ← 인메모리 버퍼 (TickBuffer 등)
└── store/       ← 상태 저장소 (analytics CandleStore)

infrastructure/
├── messaging/{consumer, publisher, config} ← Kafka adapter
├── persistence/{entity, repo, mapper, adapter} ← JPA + QueryDSL
├── cache/       ← Redis · 인메모리 adapter
└── scheduler/  ← @Scheduled adapter
```

api 모듈 — 어댑터 레이어 전용

api 모듈은 도메인 코드가 없고 어댑터 레이어만 모은 구조입니다. query 모듈을 호출하는 controller, 멀티 query 합성을 담당하는 composition, Kafka consumer + Sinks + SseEmitter를 묶은 stream layer로 분리했습니다.

```
api/
├── controller/{meta, market, analytics, economic}/
├── composition/{service, dto}/ ← 멀티 query 합성 (DashboardService)
├── stream/{config, consumer, sink, handler}/
│   └── Kafka consumer + Sinks.Many<Domain> + SseEmitter
├── common/ ← CursorPage, OffsetPage
└── config/ ← Security, OpenAPI, CORS, GlobalExceptionHandler
```

Port 인터페이스의 효용 — 실제 예시

Analytics의 파티션 lifecycle은 인터페이스 하나로 추출했습니다. Tick · Premium · PremiumDetail 세 가지 PartitionRegistry가 같은 계약을 구현하므로, Kafka consumer listener 쪽 코드가 구체 Registry 타입에 의존하지 않습니다.

```
package com.example.demo.analytics.domain.partition_registry;

public interface PartitionLifecycle {

    /** 새 파티션 할당 시 호출. 빈 상태로 초기화하거나 복원 데이터를 주입한다. */
    void assignPartition(int partitionId);

    /** 파티션 회수 시 호출. 내부 상태를 정리한다. */
    void revokePartitions(Collection<Integer> partitionIds);
}
```

설계 의도

Port를 인터페이스로 추출했기 때문에 PartitionRegistry 3종이 동일 시그니처로 통일됐고, rebalance 처리 로직이 Tick/Premium에 중복되지 않습니다. Adapter는 @Component 로 Spring이 주입하고, UseCase는 @RequiredArgsConstructor 로 생성자 주입만 사용합니다.

06. 핵심 설계 결정 5선

각 결정을 "상황 · 부딪힌 문제 · 시도 · 배움" 흐름으로 정리했습니다. 추측이나 일반론이 아니라, 실제 프로젝트를 진행하며 막혔던 지점과 그걸 어떻게 풀었는지에 대한 기록입니다.

01 아키텍처 구조 선택 — 모듈리스 + 헥사고날 + DDD

상황

실시간 시세 수집 · 환율 동기화 · 김프 계산 · 캔들 분석 · 조회 API · 실시간 스트림까지 여러 종류의 작업이 한 시스템 안에 있었습니다. 각 작업은 자원 요구량이 달랐습니다. 시세 수집은 네트워크 대역폭과 파싱 CPU가 중요하고, 분석은 메모리와 누적 연산 비용이 크고, 조회 API는 DB 응답이 병목이었습니다. 또 외부 데이터 의존성(거래소 API 스펙) 변동이 잦아 코드 변경 빈도도 영역마다 크게 달랐습니다.

부딪힌 문제

세 가지 후보를 두고 고민했습니다.

- **단일 프로젝트 + MVC**: 구조가 단순하지만 영역별 자원 격리가 안 되고, 한 영역의 변경이 다른 영역 테스트를 깨뜨림
- **마이크로서비스 (MSA)**: 자원 격리는 완벽하지만 1인 프로젝트 범위에서 운영 복잡도(서비스 디스커버리 · 분산 트랜잭션 · 모니터링)가 과함
- **모듈리스 (단일 프로세스 멀티모듈)**: 단일 프로세스의 단순함과 모듈 격리의 유연성을 동시에 가질 수 있음

시도

모듈리스를 선택하고 각 모듈 안은 **헥사고날 + DDD**로 구성했습니다. 외부 의존성(거래소 API · 메시지 큐 · 캐시 · DB)이 도메인 코드에 직접 닿지 않도록 어댑터 레이어로 격리했고, 도메인은 record 기반 불변 객체로 작성했습니다. 거래소 API 스펙이 바뀌어도 어댑터만 수정하면 도메인은 그대로 유지됩니다.

부록으로 한 가지 — 마스터 데이터 모듈은 현재 기능이 작지만 별도로 분리해두었습니다. 초기 설계에서는 거래소별 코인 전송 가능 여부 · 펀딩피 · 트레이드 규칙 등 다양한 마스터 데이터를 다룰 계획이었고, 지금은 축소돼 마켓 코드 처리만 합니다. 시세 모듈과 합칠 수도 있었지만, **다루는 데이터 성격이 시세(흐름)와 마스터(상태)로 다르고 향후 확장 시 다시 채워질 자리**이기 때문에 분리를 유지했습니다.

배움

구조 선택은 "지금 가장 옳은가"가 아니라 "**이 프로젝트가 향후 어떻게 변할 가능성이 있는가**"에 더 가까운 질문이었습니다. 외부 의존성 변동이 잦은 영역(거래소 API)을 헥사고날로 격리한 결정은, 실제로 거래소 응답 포맷이 바뀌었을 때 어댑터 한 곳만 고치고 도메인 코드는 건드리지 않아도 되는 경험으로 검증됐습니다. 반대로 모듈을 너무 잘게 쪼갠 부분(현재 23개)은 1인 프로젝트 규모에선 빌드 시간과 의존성 관리 비용이 컸고, 다시 한다면 핵심 5~6개에서 시작해 검증 후 점진 분리할 것입니다.

02 데이터 출처 결정 — 1초 캔들 API 대신 WebSocket 실시간 구독

상황

거래소는 시세 데이터를 두 가지 방식으로 제공합니다. 첫째, REST API로 "지난 1초간의 캔들 한 개"를 요청하는 방식. 둘째, WebSocket을 열어 시세가 바뀔 때마다 실시간으로 받는 방식. 구현 난이도는 REST 폴링이 훨씬 낮아서 처음에는 1초 캔들 API를 검토했습니다.

부딪힌 문제

1초 캔들 API로 받은 데이터를 기존 트레이딩 프로그램(거래소 공식 차트)의 수치와 비교해봤더니 값이 미세하게 어긋나는 현상을 발견했습니다. "1초 캔들"이라는 이름과 달리 실제로는 0.0초 · 0.2초 · 0.5초 시점의 데이터가 섞여 있는 것처럼 보였고, 고가/저가가 같은 1초 구간 안에서도 호출 시점에 따라 값이 달랐습니다. 김프 계산이나 지표 분석은 가격 한 자리 차이로 결과가 크게 흔들리는 영역이라, 이 부정확함은 무시할 수 없었습니다.

시도

WebSocket을 열어 거래소가 시세 변경을 알릴 때마다 데이터를 받는 방식으로 전환했습니다. 구현 비용은 컸지만, 받은 데이터를 거래소 공식 차트와 비교했을 때 일치했습니다. 대신 WebSocket은 연결이 끊어질 수 있는 채널이라, 재연결 로직과 누락 시점 감지를 함께 만들어야 했습니다.

배움

"공식 API라고 해서 곧 정확하다는 뜻은 아니다"라는 걸 배웠습니다. 만약 비교 기준(거래소 공식 차트) 없이 1초 캔들 API만 보고 있었다면, 부정확함을 발견하지 못한 채로 계속 진행했을 가능성이 큼니다. 선택지가 두 가지 이상 있을 때는 **둘 다 짧게 시도해보고 비교한 다음 결정**하는 게, 문서만 읽고 선택하는 것보다 훨씬 안전하다는 감각을 얻었습니다.

03 JSON 파서 교체 — 초당 3만건 환경에서 CPU 오버로 해결

상황

초기에는 자바 진영에서 가장 널리 쓰이는 표준 JSON 파서를 사용했습니다. 검증된 라이브러리이고, 학습 자료도 많고, 기본 선택으로 무난해 보였습니다.

부딪힌 문제

실제 운영 부하로 진입했을 때 시스템이 터졌습니다. 약 **1,500개 코인** × 각 코인별 초당 **10~20회**의 시세 변경, 합쳐서 **초당 15,000~30,000건의 JSON 메시지**가 들어오는 환경에서 CPU 사용률이 한계까지 치솟고 프로세스가 다운됐습니다. 프로파일링을 해보니 JSON 파싱이 압도적인 비용을 차지하고 있었습니다.

시도

파싱 방식을 다시 봤습니다. 표준 파서는 객체를 파싱할 때 reflection으로 필드를 찾아 매핑합니다. 한두 건이면 무시할 만한 비용이지만, 초당 수만 건 누적되면 CPU가 견디지 못합니다. 그래서 **reflection을 거의 사용하지 않고 컴파일 타임에 파싱 코드를 생성**하는 가벼운 파서로 교체했습니다. 교체 후 동일한 부하에서 안정적으로 동작했고, 다운 증상이 사라졌습니다.

같은 맥락에서 한 단계 더 나아간 선택지인 **이진 직렬화(Protocol Buffers 등)** 도입도 검토했습니다. JSON 자체를 텍스트가 아닌 바이너리로 바꿔 파싱 비용을 더 줄이는 방식입니다. 다만 이것 도입하려면 메시지 큐를 흐르는 모든 메시지 정의를 다시 작성해야 하고, 스키마 저장소 · 코드 자동 생성 도구 같은 인프라를 함께 갖춰야 합니다. 현재 파서 교체만으로 부하를 견디고 있고, 메시지 처리량이 한 단계 더 늘어나는 시점에 도입하는 게 맞다고 판단해 일단 보류했습니다. 대신 향후 전환을 막지 않도록, 이진 직렬화용 스키마 파일은 미리 작성해 저장소에 두었습니다(현재 미사용).

배움

"가장 많이 쓰이는 도구가 모든 상황에 맞는 건 아니다"라는 걸 직접 경험으로 배웠습니다. 그리고 도입 결정만큼 **"언제 다음 단계로 갈 것인가"의 기준**을 미리 정해두는 것도 중요했습니다. 지금은 가벼운 파서로 충분하지만, 처리량이 또 한 단계 늘면 이진 직렬화로 가는 것이 다음 단계라는 게 명확하게 그려져 있습니다.

한 가지 아쉬운 점은 당시 CPU 사용률 변화의 정확한 수치를 측정 기록으로 남기지 않은 것입니다. 지금이라면 마이크로벤치마크 도구로 단일 메시지 파싱 비용을 측정해 정량적 근거를 함께 남겼을 것입니다.

04 모든 시세를 저장하지 않는다 — 저장 범위와 도메인 분리 결정

상황

결정 3의 문제를 풀고 나서 새로운 문제가 보였습니다. 초당 수만 건의 시세를 다 받을 수 있게 됐지만, 그걸 다 저장하면 DB 용량과 디스크 비용이 감당이 안 됐습니다. 김프 계산에 필요한 건 "지금 이 순간의 가격"이지, 1초 전 가격 1만 개가 아닙니다.

부딪힌 문제

시세를 다 저장하면: DB가 빠르게 차고, 1인 프로젝트 예산을 초과합니다. 시세를 안 저장하면: 나중에 "그 시점의 가격이 얼마였느냐"를 추적할 수 없습니다. 둘 사이의 절충점을 찾아야 했습니다.

시도

시세 자체는 저장하지 않고, **계산된 결과물(프리미엄 · 캔들 · 지표)만 저장**하기로 했습니다. 실시간 화면에 보여줘야 하는 "현재 가격"은 외부 캐시에 짧은 만료 시간으로 보관해 DB까지 가지 않고도 응답할 수 있게 했습니다.

같은 결정의 다른 측면 — 김프 데이터를 **두 가지 형태로** 분리해 저장한 점도 있습니다. 하나는 "일반 사용자가 보는 김프"(국내 가격 대비 해외 가격이 몇 % 비싼지), 다른 하나는 "환헷지를 고려한 김프 상세"(그 시점의 환율 · 양쪽 호가 · 헷지 비용까지 포함).

일반 사용자 화면에서는 단순한 김프 수치가 익숙하고, 다른 사이트와 비교하기도 편합니다. 하지만 실제 김프 투자자에게는 "**이 때 환율이 얼마였느냐**"가 결정적입니다. 환율 변동을 헷지하지 못한 상태에서는, 김프가 떨어진 게 가격 차이 축소 때문인지 환율 변동 때문인지 구분이 안 되면 의사결정을 할 수 없습니다. 반대로 기술 지표(이동평균 · RSI 등)를 만들 때는 환율 요소가 노이즈로 작용하므로 단순 김프 수치가 더 적합합니다. 같은 데이터를 두 가지 시각으로 보기 위해 분리한 결정이었습니다.

배움

"**저장할 수 있다**"와 "**저장해야 한다**"는 다른 질문이라는 걸 배웠습니다. 예산과 자원이 무한하다면 다 저장하는 게 미래 옵션을 넓히지만, 실제로는 "이걸 다 저장해서 어떤 가치를 만들 것인가"를 먼저 답할 수 있어야 저장이 정당화됩니다. 지금은 데이터 제 공업이나 정밀 투자 분석으로 확장할 가치가 명확해질 때 다시 결정할 항목으로 남겨두었습니다.

그리고 도메인 모델을 만들 때는 "**이 데이터를 누가, 어떤 의사결정을 위해 보는가**"라는 질문이 구조 설계의 출발점이라는 감각도 얻었습니다. 같은 원시 데이터라도 사용자가 다르면 저장 형태도 달라야 합니다.

05 Redis 채택과 분산 처리 상태 복원 — 인메모리와 DB 사이의 절충

상황

시세는 매번 들어오는 즉시 화면에 보여줘야 하고, 캔들 계산 같은 작업은 누적 상태를 어딘가에 보관해야 했습니다. "어디에 보관할 것인가"가 필요했습니다. 세 가지 후보가 있었습니다. **JVM 인메모리**(가장 빠름, 휘발성), **DB**(영속성 확보, 지연 큼), **Redis**(인메모리 속도에 가깝되 외부 저장).

부딪힌 문제

인메모리만 쓰면 프로세스가 죽거나 분산 처리에서 작업이 재분배될 때 상태가 통째로 사라집니다. 캔들 계산처럼 누적 상태가 5분치 쌓여 있으면, 사라진 순간 그 5분 봉은 영원히 망가집니다. 그렇다고 DB에 매번 쓰는 건 응답 시간이 너무 길고, 거래소가 추가될수록 데이터량이 선형으로 늘어나 DB 부담이 비례 증가합니다.

시도

Redis를 선택하고 두 가지 용도로 나눠 썼습니다.

- **실시간 조회용**: "지금 가장 최근 가격"처럼 화면에 즉시 보여줘야 하는 값. 짧은 만료 시간으로 보관.
- **상태 복원용**: 캔들 · 지표의 누적 계산 상태. 주기적으로 스냅샷을 저장해두고, 작업이 다른 인스턴스로 재할당될 때 캐시에서 복원.

두 번째 용도가 까다로웠습니다. 분산 메시지 처리에서는 인스턴스가 추가/제거될 때마다 작업이 자동 재분배되는데, 그 시점에 메모리 상태가 사라집니다. 두 가지 복원 방식을 비교했습니다. 하나는 메시지 큐의 과거 기록을 다시 읽어 처음부터 누적을 재계산하는 방식. 다른 하나는 Redis 스냅샷에서 읽어 즉시 주입하는 방식. 전자는 쌓인 메시지가 많을수록 복원이 느려지지만, 후자는 인스턴스 개수만큼만 읽으면 끝나서 **일정한 시간 안에 복원**이 끝납니다. 실시간 처리에서 멈춤 시간이 짧은 게 더 중요하다고 판단해 후자를 택했습니다.

대신 작업 재할당 시점에 호출되는 "할당됨" / "회수됨" 두 가지 신호를 인터페이스로 먼저 정의했습니다. 시세 · 김프 · 김프 상세 세 종류의 처리기가 모두 같은 인터페이스를 구현하므로, 메시지 처리 쪽 코드는 어떤 종류의 처리기인지 신경 쓰지 않아도 됩니다.

배움

"성능"과 "영속성" 사이의 절충점을 고를 때, "**지금 가장 빠른 것**"이 아니라 "**데이터량이 늘어났을 때도 견디는 것**"을 우선했습니다. 거래소가 추가되거나 추적 종목이 늘어났을 때 부작용이 가장 적은 선택을 한 것입니다.

지금 구조의 약점은 **Redis가 죽으면 상태 복원 경로 자체가 없다**는 점입니다. 다음 보강에서는 Redis가 안 될 때 메시지 큐 재생으로 우회하는 안전망을 추가할 계획입니다. "이상적인 해결책"보다 "**현재 환경에서 가장 빠른 복원**"을 우선했고, 그 트레이드오프를 문서에 명시했습니다.

다섯 결정의 공통 원칙

"지금 환경에 맞는 선택을 하되, 다음 단계로 갈 기준을 함께 정한다."

모든 결정에 "왜 지금은 이것 선택하는가"뿐 아니라 "언제 다음 단계로 갈 것인가"의 기준이 함께 있습니다. JSON 파서는 이진 직렬화로, Redis는 메시지 큐 재생 안전망으로, 모듈 구조는 점진적 분리 — 다음 단계를 막지 않도록 코드와 문서에 흔적을 남겼습니다. 완벽한 코드보다 다음 사람(또는 미래의 나)이 이어가기 쉬운 코드를 목표로 했습니다.

07. 협업 · 범위 관리 결정

1인 프로젝트지만 "혼자 짤 코드"가 아닌 **"다른 사람이 이어받을 수 있는 코드"**를 목표로 했습니다. 실제 팀에서 일어나는 두 가지 협업 상황을 가정하고 그 안에서 의사결정했습니다.

A Trading 모듈을 넣지 않은 진짜 이유 — 기술 구조와의 충돌

상황

프로젝트 초기 기획에 자동매매(Trading) 모듈이 포함돼 있었습니다. 김프를 실시간으로 추적하는 시스템이므로, 그 위에 자동 주문까지 없는 건 자연스러운 확장이었습니니다. "안 넣으면 반쪽짜리 프로젝트가 되지 않을까" 하는 부담도 있었습니다.

부딪힌 문제

막상 설계에 들어가니 단순한 "기능 추가"가 아니라 **전체 구조와 충돌하는 결정**이라는 게 드러났습니다. 리얼 트레이딩은 시세가 들어온 그 순간의 가격으로 주문을 넣어야 합니다. 몇 백 밀리초의 지연도 의도와 다른 가격에 체결됩니다. 그러려면 트레이딩 코드는 시세 수집 코드와 **같은 메모리 공간에서 즉시 데이터를 받아야** 합니다. 메시지 큐를 거치는 순간 그 지연이 발생합니다.

그런데 현재 구조는 시세 수집(ingestion) → 메시지 큐 → 김프 계산 → 분석으로 흐릅니다. 모든 모듈이 메시지 큐를 통해 느슨하게 연결되어 있어 각자 독립적으로 확장 가능한 구조입니다. 트레이딩을 넣으려면 이걸 깨고 **시세 수집 모듈 안에 트레이딩 코드를 합치는 결정**이 필요했고, 그건 지금 만든 모듈 분리 원칙을 뒤집는 일이었습니다.

시도

두 가지 선택지를 두고 결정해야 했습니다. 첫째, 핵심 데이터 파이프라인 구조를 깨고 트레이딩을 끼워 넣기. 둘째, 트레이딩을 빼고 데이터 파이프라인을 깊이 안정화하기.

둘 다 어중간하게 가는 건 최악의 선택이라고 판단했습니다. 파이프라인이 안정되지 않은 채로 트레이딩을 넣으면, 자동 주문이 잘못된 데이터로 돌아가 실제 금전 피해로 이어질 수 있습니다. 반대로 트레이딩만 잘 만들고 데이터가 부정확하면 그것대로 의미가 없습니다.

그래서 **데이터 파이프라인을 끝까지 안정화**하는 쪽을 택하고, 트레이딩은 모듈 자리만 잡아둔 채 코드는 비워두었습니다. 문서에는 "거래소 인증 · 리스크 제어 · 핫 데이터 경로 재설계가 함께 필요하므로 핵심 파이프라인 안정화 후 단계"라고 명시했습니다.

배움

"기능을 빼는 결정"이 "기능을 넣는 결정"보다 어려울 수 있다는 걸 배웠습니다. 넣는 건 시간만 들이면 되지만, 빼는 건 "왜 빠졌느냐"는 질문에 매번 답해야 하기 때문입니다. 그래서 "왜 안 했나"의 답을 코드 구조와 연결해 설명할 수 있도록 문서로 남겼습니다. 욕심나는 기능을 다 넣으려다 "다 어중간한 프로젝트"가 되는 것보다, **핵심을 깊이 파고 나머지는 명시적으로 미루는 쪽**이 팀에서 일할 때 더 신뢰감을 준다는 걸 배웠습니다.

B 코드에 남은 오타를 즉시 고치지 않고 별도 작업으로 분리

상황

프로젝트 진행 중 패키지 이름과 클래스 이름에 오타가 여러 개 있다는 걸 발견했습니다. "analytics"가 "analystics"로, "ingestion"이 "injection"으로, "infra"가 "infre"로 적힌 곳들이 있었습니다. 보기 싫고 즉시 고치고 싶었습니다.

부딪힌 문제

오타를 고치려면 수십 개 파일의 임포트 경로를 한꺼번에 바꿔야 했습니다. 그런데 같은 시기에 새 기능 개발도 진행 중이었습니다. 오타 수정과 기능 개발이 같은 변경 단위에 섞이면, 나중에 변경 이력을 봤을 때 "무엇이 기능 변경이고 무엇이 단순 이름 변경인지" 구분이 안 됩니다. 뭔가 잘못됐을 때 어디로 되돌려야 할지 추적이 어려워집니다.

시도

오타는 그대로 두고 계속 진행하기로 했습니다. 새로 추가하는 코드도 주변 모듈의 기존 오타 패턴을 따랐습니다. 대신 문서에 **"이건 의식적으로 유지 중인 오타이며, 별도 정리 작업에서 일괄 변경해야 한다"**고 목록으로 명시했습니다. 신규 합류자가 "왜 이런 이름이 있지?" 헛갈리지 않도록 한 것입니다.

배움

"한 번에 한 가지만 바꾼다"는 원칙은 코드 리뷰 효율과 안정성에 직결됩니다. 보기 싫다는 이유로 기능 변경에 리팩토링을 끼워 넣으면, 리뷰어가 변경의 의도를 파악하기 어려워지고 문제가 생겼을 때 원인 추적이 길어집니다. 혼자 짤 코드에서도 **"미래의 내가 이 변경을 이해할 수 있는가"**라는 관점에서 같은 원칙을 적용했고, 실제로 몇 달 후 다시 봐도 변경 이력이 깔끔하게 읽히는 걸 경험했습니다.

08. 코딩 컨벤션 · 코드 품질

23개 모듈에 걸쳐 코드가 흩어져 있어도 같은 패턴으로 읽히도록, 작은 단위 규칙을 일관되게 적용했습니다. 아래는 그중 면전에서 자주 확인되는 핵심 컨벤션입니다.

제네릭 타입 파라미터 네이밍

매퍼·버퍼·캐시·포트 시그니처에 등장하는 제네릭 파라미터를 6가지 키워드로 통일. 어느 모듈에서든 `<DOMAIN, ENTITY>` 를 보면 의도를 한 번에 읽을 수 있도록 했습니다.

파라미터	의미	예시
DOMAIN	도메인 객체	Tick, Premium, TickCandle
MESSAGE	Kafka 메시지 record	TickMessage, PremiumMessage
ENTITY	JPA Entity	TickEntity, MarketCodeEntity
KEY	캐시 · 맵 키	TickKey, PremiumKey, FxKey
VAL	값 객체	BigDecimal, PremiumDetailValue
RAW	원시 데이터	String, JsonNode

매퍼 인터페이스 — 변환 방향을 타입으로 강제

"원시 데이터 → 메시지 → 도메인 → 엔티티" 4단 변환을 8가지 매퍼 인터페이스로 표준화했습니다. 개발자가 변환 함수를 새로 만들 때마다 매번 시그니처를 고민하지 않고, "어느 계층에서 어느 계층으로 가는가"만 정하면 매퍼 종류가 자동으로 결정됩니다. 공통 모듈 한 곳에서 관리하므로 변환 패턴이 모듈마다 제각각이 되는 것을 방지합니다.

Spring Bean 규칙

- **의존성 주입** 생성자 주입만 사용. 필드 주입과 setter 주입은 금지
- **불변 객체** 우선 도메인은 자바 record로 정의, 상태가 변하는 객체만 클래스로 작성
- **설정과 빈 정의 분리** 환경 설정은 `@Configuration` 으로 격리

네이밍 표준

KAFKA 토픽

`{소스모듈}.{데이터타입}` 패턴으로 통일.

예: `ingestion-exchange.tick-raw`, `market-data.premium`, `analytics.tick-candle`. 토픽 이름만 봐도 어느 모듈이 발행했는지 알 수 있습니다.

REDIS 키

공통 접두사 `ys:{env}:v1:` 로 환경별 충돌 방지.

키 생성은 전용 유틸리티 클래스로만 가능하게 강제. 문자열 하드코딩을 금지해서 키 구조 변경 시 한 곳만 수정하면 됩니다.

보안 · 시크릿 관리

설정 파일에 시크릿을 직접 노출하지 않고 환경 변수 fallback만 사용합니다. 테스트 설정도 placeholder 처리하여 실수로 키가 저장소에 올라가는 사고를 차단했습니다. API 인증은 현재 비활성 상태이며, 외부 클라이언트 단계 진입 시 토큰 기반 인증을 추가할 예정입니다.

09. 테스트 전략

레이어별로 적절한 슬라이스를 선택해 검증합니다. UseCase는 Mock 기반 단위 테스트, 어댑터는 WireMock/Testcontainers 통합 테스트, Controller는 @WebMvcTest 슬라이스, SSE는 StepVerifier 로 검증.

레이어별 테스트 전략

레이어	슬라이스 / 도구	검증 항목
UseCase	JUnit 5 + BDD Mockito (given/ then)	반환값 · Port 호출 인자(ArgumentCaptor) · 호출 횟수 · InOrder 흐름
Domain / Buffer / Store	POJO 단위 테스트	BigDecimal 정밀도, ConcurrentHashMap 동시성, snapshot → clear race window
JPA Adapter	@DataJpaTest + Testcontainers PostgreSQL	매퍼 라운드트립, QueryDSL 쿼리, 시간 범위 검증
Kafka Publisher / Consumer	Testcontainers Kafka 또는 @EmbeddedKafka	토픽 상수 참조, payload 라운드트립, 직렬화 호환성
외부 HTTP (FRED · Naver · Investing)	WireMock + fixture JSON/HTML	파싱 정확성, 페이지 구조 변경 시 selector 강건성
Controller	@WebMvcTest + @MockBean	RFC 7807 ProblemDetail, cursor/offset 페이징, 시간 epoch ms
SSE Handler	StepVerifier + Sink 직접 emit	도메인 객체 Sink 시그니처, Disposable 해제, onComplete/onTimeout/onError
Architecture	ArchUnit (권장 단계)	api → 도메인 모듈 의존 0건, composition → 도메인 호출 0건

비동기 검증

Thread.sleep 는 금지. **Awaitility**로 polling-with-timeout 패턴을 통일했습니다. Kafka publish → DB write 같은 비동기 흐름은 outbox state 변화나 row count로 검증합니다.

BDD Mockito 통일

```
// Mockito.when 대신 BDDMockito.given 사용으로 BDD 흐름 통일
given(exchangeGetter.get(20L)).willReturn(Optional.of(ex("Binance", "USDT")));
given(fxGetter.get(new FxKey("USD", "KRW")))
    .willReturn(Optional.of(fx("USD", "KRW", "0.000725")));

sut.cal(1L);
sut.cal(2L);

then(buffer).should(times(1)).add(premiumCaptor.capture());
then(premiumPublisher).should(times(1)).process(premiumCaptor.getValue());

Premium premium = premiumCaptor.getValue();
assertThat(premium.symbol()).isEqualTo("BTC");
```

모듈 우선순위 (테스트 보강 로드맵)

1. market_data 완료
2. analytics 완료
3. meta_data 완료
4. ingestion/* (exchange + fx) 완료
5. economic/* (shard + fred + crawling) 완료
6. api 완료
7. query/* 4종 SQL 실행 통합 테스트 보강 후보

10. 한계 · 학습 · 향후 작업

포트폴리오 가치를 위해 결함을 감추기보다, 인지한 한계를 정직하게 정리합니다. "안 한 것"과 "못 한 것"을 구분하고, 향후 작업의 우선순위를 명시했습니다.

의도적 MVP 제외 항목 (안 한 것)

항목	제외 이유
Trading 모듈 (자동매매, 주문, 포지션, 리스크)	핫 데이터 결합 요구로 ingestion 모듈 재설계가 함께 필요. 핵심 파이프라인 안정화 이후 단계로 미룸. 상세는 섹션 07-A 참조
이진 직렬화 (Protocol Buffers 등)	현재 파서 교체만으로 부하 건디는 중. 메시지 처리량이 한 단계 더 늘어나는 시점에 도입 예정. 스키마 파일은 미리 작성해 보관(현재 미사용). 상세는 섹션 06-③ 참조
모든 시세 원시 데이터 저장	DB 용량과 비용 부담. 정밀 투자 분석이나 데이터 제공업으로 확장 시 재검토 예정. 상세는 섹션 06-④ 참조
경제지표 실시간 downstream	수집 · 저장 · REST 조회는 완료. analytics 연계 · SSE 연결은 발표 주기가 market data와 달라 MVP 데모 범위 밖.
프론트엔드 대시보드	별도 프로젝트 <code>coin_front</code> (Next.js 15 / React 19)에서 담당. API 응답 컨벤션은 양쪽 합의 완료.
인증(JWT) · 알람 · 워치리스트 백엔드	현재 <code>SecurityFilterChain</code> 이 <code>permitAll()</code> . 외부 클라이언트 단계에서 추가. <code>coin_front</code> 는 mock 데이터로 화면 시연 가능.

인지한 약점 (못 한 것 · 보강 예정)

운영 · 트래픽 실측 부재

로컬 Docker로 전체 파이프라인 검증. EKS/GKE 클라우드 배포와 합성 데이터 외 실측 throughput/latency 수치 없음. retention.bytes 같은 수치는 예상 트래픽 기반 추정값.

REDIS 단일 장애점

analytics state 복원이 Redis에만 의존. Kafka replay fallback(analytics.* 24h) 미구현. 다음 보강 1순위.

인증 · CI/CD 미완성

`SecurityFilterChain` 이 `permitAll()` 상태. 외부 클라이언트 단계에서 JWT/API Key 추가 예정. GitHub Actions 자동화도 동시 작업 대기.

QUERY SQL 통합 테스트 갭

query/* 4종에 mapper · 파라미터 단위 테스트는 있음. Testcontainers 기반 실제 PostgreSQL 실행 검증 추가 필요.

향후 작업 우선순위

1. Redis 장애 대비 fallback — analytics.* 24h Kafka replay 경로 구현
2. ArchUnit 가드 도입 — write/read 분리, api 의존성 경계를 컴파일 가까이 검증
3. query 모듈 Testcontainers 통합 테스트 — 실제 SQL 실행 검증
4. 인증 · 인가 + CI/CD — JWT/API Key, GitHub Actions 자동화
5. analytics outbox 패턴 — DB write 후 Kafka publish 실패 시 재시도 보장

이 프로젝트로 얻은 학습

- 구조 선택은 "지금 가장 옳은가"가 아니라 "이 프로젝트가 어떻게 변할 가능성이 있는가"에 가까운 질문
- "가장 많이 쓰이는 도구가 모든 상황에 맞는 건 아니다" — 실측 부하에서 측정하고 가설을 검증해야 진짜 결정이 됨
- "저장할 수 있다"와 "저장해야 한다"는 다른 질문 — 예산과 자원이 무한하지 않다면 가치 정의가 먼저
- "기능을 빼는 결정"은 "넣는 결정"보다 어렵다 — 빼는 이유를 코드 구조와 연결해 설명할 수 있어야 함
- "안 한 것"을 명시적으로 문서화하면 면접/리뷰에서 방어가 아니라 협업의 대화가 가능

11. 차별점 요약

수많은 "메시지 큐와 스프링을 써본 사이드 프로젝트"와 이 프로젝트를 구분 짓는 지점입니다.

기술적 차별점

① 실측 부하에서 시스템이 터진 경험과 그 해결

초당 15,000~30,000건의 JSON 메시지가 들어오는 환경에서 CPU 오버로 프로세스가 다운되는 증상을 만났고, 파싱 방식을 reflection 기반에서 컴파일 타임 코드 생성 방식으로 바꿔 해결했습니다. **"라이브러리를 골랐다"가 아니라 "부하 환경에서 측정하고, 가설 세우고, 검증해서 바꿨다"**는 흐름의 경험입니다. 측정 기록을 남기지 않은 점은 아쉽고, 지금이라면 마이크로벤치마크로 정량적 근거를 함께 남겼을 것이라고 정직하게 인정합니다.

② 구조 선택에 후보를 비교한 흔적이 남아 있음

모듈리스 · 멀티모듈 · MSA 세 가지를 진지하게 검토하고 각각의 트레이드오프를 비교한 끝에 모듈리스 + 헥사고널 + DDD를 선택했습니다. **"기본 선택을 따랐다"가 아니라 "왜 이걸 골랐는지"**에 답이 있는 결정이고, 외부 데이터 의존성이 잦게 변하는 환경에서 어댑터 격리가 실제로 어떻게 작동하는지 경험으로 검증했습니다.

③ 도메인을 사용자 관점으로 분리한 결정

김프 데이터를 "일반 사용자용 단순 김프"와 "투자자용 김프 상세(환율 · 호가 · 헷지 비용 포함)" 두 가지로 분리했습니다. **"같은 데이터를 누가 어떤 의사결정을 위해 보는가"**를 도메인 설계의 출발점으로 삼은 결정으로, 김프 투자의 실제 워크플로(환헷지 유무에 따라 분석 관점이 달라짐)를 이해하고 만든 구조입니다.

협업 관점에서의 차별점

④ "기능을 빼는 결정"을 코드 구조와 연결해 설명

Trading 모듈을 뺀 이유가 "못 해서"가 아니라 "한 데이터 결합이 필요해서 ingestion 모듈 재설계까지 필요한데, 그건 핵심 파이프라인 안정화 이후 단계"라는 점. **안 한 것에 대한 답이 코드 구조와 연결되어 있어**, 면접 자리에서 "이 기능은 왜 없어요?"에 명확한 답이 가능합니다.

⑤ "한 번에 한 가지만 바꾼다" — 리팩토링과 기능 개발을 분리

프로젝트 중간에 패키지 이름 오타를 발견했지만 즉시 고치지 않았습니다. 이름 변경은 임포트 수십 개를 한꺼번에 건드리므로, 기능 개발과 같은 단위에 섞이면 변경 이력이 뒤죽박죽이 됩니다. 혼자 짜는 코드에서도 **"미래의 내가 이 변경을 이해할 수 있는가"**라는 관점에서 같은 원칙을 적용했습니다.

이 포트폴리오에서 보여드리고 싶은 것

SUMMARY

- 측정하고, 가설 세우고, 검증한다.** 라이브러리를 고를 때도 구조를 정할 때도 "기본 선택"을 따르기보다 후보를 비교해보고 결정합니다.
- "지금 환경에 맞는 선택"과 "다음 단계로 갈 기준"을 함께 정한다.** 모든 결정에 "왜 지금은 이걸 고르는가"와 "언제 다음 단계로 갈 것인가"가 함께 있습니다.
- 한계를 인정하는 정직함.** 모르는 것을 모른다고 말하고, 못 한 것은 "다음에 보강할 항목"으로 정리합니다.

Thank you. — End of Portfolio.